

Artificial Intelligence — Semester Project

A Simple Network Intrusion Detection System

Author: Saad Ahmad

Total Marks: **20** | Team Size: **1–2 students**

1 Objective

The goal of this project is to give you hands-on practice with the AI and machine-learning techniques we have covered in class by building a simple **Network Intrusion Detection System (NIDS)**. Given a set of features describing a network connection, your system must decide whether the connection is *normal* or an *attack*.

By the end of the project you should be comfortable with:

- Loading and exploring a tabular dataset with **pandas**.
- Training, testing and comparing classical ML models in **scikit-learn**.
- Visualising results with **matplotlib**.
- Designing a very small *intelligent agent* and a basic *genetic algorithm* in plain Python.

2 Problem Description

Network administrators receive thousands of connection records every minute. Inspecting every record by hand is impossible, so we want a program that can look at a connection and **predict whether it is an attack**.

You are provided with a cleaned, simplified dataset (`dataset/network_traffic.csv`) containing **6,000** connection records. Each row has:

- **15 numeric features** that describe the connection (duration, bytes transferred, error rates, counts of similar connections, etc.). See `dataset/README.md` for the meaning of every column.
- A single target column `label` where 0 = **normal** and 1 = **attack**.

The classes are balanced (3,000 of each). All features are already numeric, so **no label-encoding or one-hot encoding is required**. There are no missing values.

3 Tasks

The project has five tasks. Complete them in order. Every task should be in your notebook.

Task 1 — Data Exploration

(1 mark)

- Load the CSV with **pandas** and print the shape, the first few rows, and `df.describe()`.
- Plot the **class distribution** (a bar chart of the `label` column).

- Plot at least **two feature histograms** (e.g. `src_bytes` and `count`) with one histogram per class on the same axes so you can see where normal and attack traffic differ.

Task 2 — Simple Reflex Agent (rule-based baseline) (2 marks)

A simple reflex agent picks an action based only on the current percept using if-then rules. Build such an agent as a baseline classifier:

- Look at the histograms from Task 1 and choose **two or three features** that seem to separate the classes (for example `error_rate`, `same_srv_rate`, `count`).
- Write a Python function `reflex_agent(row)` that returns 0 or 1 using a small set of hand-picked thresholds, e.g.
$$\text{if } \text{error_rate} > 0.5 \text{ then } 1, \text{ else } 0.$$
- Apply the agent to the *whole* dataset and report **accuracy** and the **confusion matrix**.
- In a short paragraph in your report, explain why this is an example of a *simple reflex agent* (percept $\rightarrow$ rule $\rightarrow$ action, no internal state).

Task 3 — Supervised Learning (4 marks)

Split the data **once** with `train_test_split(..., test_size=0.2, random_state=42, stratify=y)` and reuse that split for every classifier below.

Train and evaluate the following three classifiers from `scikit-learn` on the same split:

1. **K-Nearest Neighbors** (`KNeighborsClassifier`). Try $k \in \{1, 3, 5, 7, 9, 11\}$ and pick the best k using the test accuracy. Plot test accuracy vs. k .
2. **Naïve Bayes** (`GaussianNB`).
3. **Logistic Regression** (`LogisticRegression`). Set `max_iter=1000`.

For each classifier report **accuracy**, **precision**, **recall**, **F1-score** and the **confusion matrix**. Put all results in a single comparison table in your report.

Hint: Some models (KNN, Logistic Regression) work much better when features are on a similar scale. Use `StandardScaler` (fit it on the training set only). Naïve Bayes does not need scaling.

Task 4 — Clustering with K-Means (2 marks)

Pretend for a moment that you do *not* have the labels:

- Drop the `label` column and scale the features.
- Fit `KMeans(n_clusters=2, random_state=42, n_init=10)`.
- Compare the cluster IDs to the true labels using a confusion matrix. (It is fine if the cluster numbering is flipped; that is expected.)
- Use PCA with `n_components=2` *only for visualisation* and plot a 2D scatter plot coloured first by the true label, then by the cluster ID. Put the two plots side by side.

Task 5 — Genetic Algorithm for Feature Selection**(1 mark)**

Design a tiny genetic algorithm (GA) that searches for a subset of features that maximises classification accuracy.

- A chromosome is a binary vector of length 15 (one bit per feature). A 1 means “use this feature”.
- The fitness of a chromosome is the test accuracy of `LogisticRegression` trained on only the selected features.
- Use a small population (e.g. 10 chromosomes) and run only a few generations (e.g. 10). Implement selection, one-point crossover, and bit-flip mutation *yourself* in plain Python — do **not** use a GA library.
- Report the best subset found and compare its accuracy to using all features. Briefly discuss whether the reduced model is faster / simpler.

This task is intentionally short. It is your chance to show you understand how GAs work.

4 Implementation Guidelines

- Use **Python 3** with `numpy`, `pandas`, `scikit-learn`, and `matplotlib` only.
- Use `train_test_split` from `sklearn` for Task 3 and Task 5 with `random_state=42`.
- All plots must use `matplotlib`. Give every plot a title and axis labels.
- Your code should run by running all cells in a jupyter notebook without manual tweaking.
- Keep your code clean: meaningful variable names, short functions, and short comments where the intent is not obvious.

5 Practical Tips

- **Start with the data.** Before coding any model, spend 30 minutes plotting histograms and looking at `df.describe()`. You will understand the problem much faster this way.
- **Pick the right metric.** Accuracy is fine here because the classes are balanced, but always also look at the confusion matrix — an NIDS that never flags attacks (all zeros) can still get 50% accuracy, which is useless.
- **Scale your features** before using KNN and Logistic Regression. Otherwise `src_bytes` (thousands) will dominate `serror_rate` (between 0 and 1).
- **Keep the split fixed.** Use the same `random_state=42` everywhere so your numbers are reproducible.
- **Write the report as you code.** Every time you make a plot or a table, paste it into the report right away. Do not leave writing to the last night.
- **Do not just paste code from an AI tool.** You will have to explain and run every part of your solution in the *viva*. Partial, honest work that you understand is worth far more than a perfect-looking submission you cannot defend.

6 Marking Scheme

Component	Description	Marks
Implementation	Tasks 1–5 correctly implemented and running	10
	Task 1: Data exploration + plots	1
	Task 2: Simple reflex agent	2
	Task 3: KNN + Naïve Bayes + Logistic Reg.	4
	Task 4: K-Means clustering	2
	Task 5: Genetic algorithm for feature sel.	1
Report	Clarity, structure, plots, analysis (see below)	10
	Introduction & problem formulation	2
	Method description for each task	3
	Results (tables, plots, confusion matrices)	3
	Discussion & conclusion	2
Total		20

7 Team Policy

- Teams of **1 or 2 students. No exceptions.**
- Both members must be able to explain every part of the submission in the viva. If one student cannot explain a task, that task may be marked as 0 for both members.
- You may discuss *ideas* with other teams, but the submitted code and report must be your own.

8 Deliverables

Submit a single ZIP file named `AI_Project_<roll1>_<roll2>.zip` containing:

- Your code (`.py` files or a single Jupyter notebook).
- A PDF of your report (4–6 pages).
- A short `README.md` explaining how to run your code.
- **Do not** include the dataset in the ZIP; we already have it.